RELATÓRIO TÉCNICO: TESTES DE INTEGRAÇÃO DO SISTEMA DE AGENDAMENTO DE VOOS OFFSHORE (AEROLOG)

Professor: Hugo Menezes Barra

Aluno: Jamil Cherem Schneider Filho

Curso: Análise e Desenvolvimento de Sistemas

Data: 29/09/2026

Repositório GitHub: https://github.com/jamilcsf/projeto-teste-integracao-helicoptero.git

1. Introdução e Contextualização do Case

A operação de transporte aéreo offshore por helicópteros exige níveis críticos de segurança, confiabilidade e precisão no agendamento de voos. Falhas na validação de disponibilidade de aeronaves, agendamentos duplos para o mesmo horário ou autorização de voos para aeronaves sob manutenção podem resultar em paralisações operacionais graves, prejuízos financeiros substanciais e risco à vida humana.

No contexto do sistema **AeroLog**, os testes de integração desempenham um papel fundamental. Ao contrário dos testes unitários — que avaliam funções de forma isolada —, os testes de integração garantem que o fluxo completo da regra de negócio funciona perfeitamente na prática: desde a chamada do serviço principal (BookingService), passando pelas regras de validação (ValidationModule), até a persistência real dos dados no banco de dados SQLite (Database).

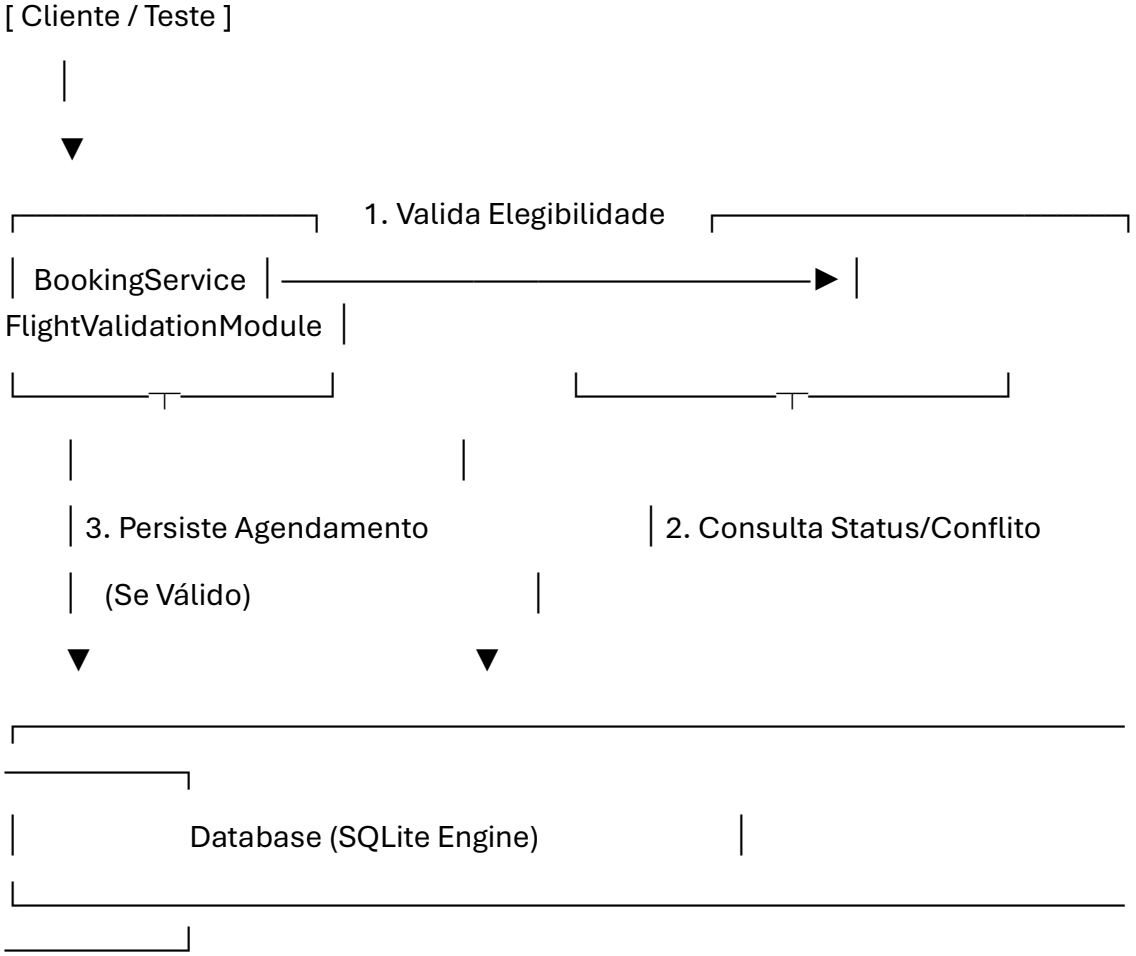
Ao utilizar uma abordagem de testes de integração **sem mocks**, assegura-se que as consultas SQL, restrições de chaves primárias/estrangeiras e o isolamento de transações comportam-se exatamente como esperado em ambiente de produção.

2. Arquitetura da Solução e Integrações

A arquitetura do módulo de agendamento é estruturada em três camadas integradas:

1. BookingService **(Camada de Orquestração):** Recebe as requisições de agendamento de voo, coordena o fluxo de validações prévias e comanda a persistência no banco de dados.
2. FlightValidationModule **(Camada de Regra de Negócio):** Consulta o banco de dados para verificar a elegibilidade da aeronave (se está cadastrada e com status LIBERADO) e garante a ausência de conflitos de horário.
3. Database **(Camada de Persistência):** Gerencia a conexão com o banco de dados SQLite e executa operações DDL/DML, como criação de tabelas, inserção e consulta de aeronaves e agendamentos.

Plaintext



3. Matriz de Cenários de Testes

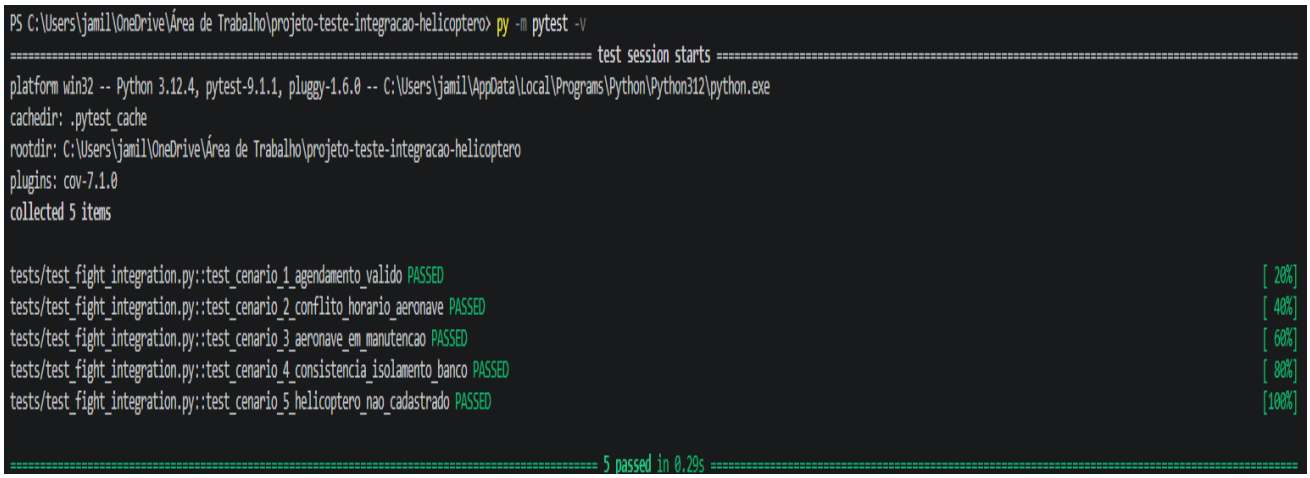
ID	Cenário de Teste	Dados de Entrada	Comportamento Esperado	Resultado Obtido
CT-01	Agendamento Válido (Caminho Feliz)	Helicóptero: "PR-AER" (LIBERADO) Horário: "2026-09-30T10:00:00" Passageiros: 4	Registro criado com sucesso na tabela agendamentos com status "AGENDADO".	**APROVADO** (PASS)
CT-02	Conflito de Horário da Aeronave	Helicóptero: "PR-AER" Horário: "2026-09-30T14:00:00" (já agendado)	Lançamento de ValueError informando conflito de horário. Nenhum registro duplicado inserido.	**APROVADO** (PASS)
CT-03	Aeronave em Manutenção	Helicóptero: "PR-MNT" (EM_INSPECAO) Horário: "2026-09-30T16:00:00"	Lançamento de ValueError indicando aeronave em manutenção. Agendamento recusado.	**APROVADO** (PASS)

ID	Cenário de Teste	Dados de Entrada	Comportamento Esperado	Resultado Obtido
CT-04	Isolamento e Consistência do Banco	Consulta direta de contagem nas tabelas agendamentos e helicopteros ao iniciar a *fixture*.	Retornar contagem 0 para ambas as tabelas, confirmando rollback/limpeza entre testes.	**APROVADO** (PASS)
CT-05	Aeronave Não Cadastrada	Helicóptero: "PR-INEXISTENTE" Horário: "2026-09-30T10:00:00"	Lançamento de ValueError indicando que a aeronave não está cadastrada.	**APROVADO** (PASS)

4. Evidências de Execução e Cobertura

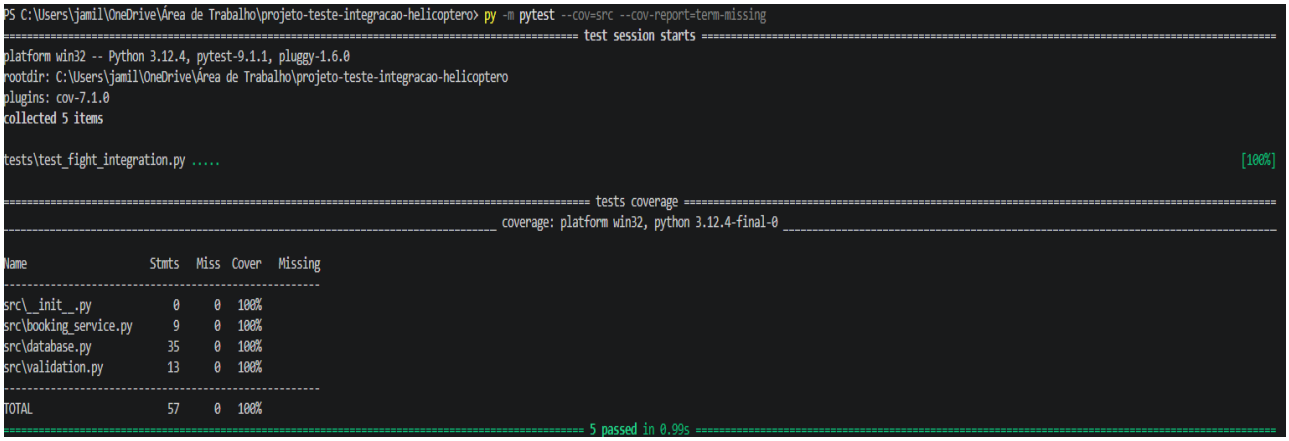
4.1. Execução dos Testes (pytest -v)

Abaixo apresenta-se a captura de tela do terminal demonstrando a execução detalhada e aprovação de todos os 5 cenários de teste de integração programados no pytest:



4.2. Relatório de Cobertura de Código (pytest-cov)

Abaixo apresenta-se a captura de tela do relatório de cobertura de código emitido pela biblioteca pytest-cov, comprovando 100% de cobertura sobre todos os módulos da camada de código-fonte (src/):



5. Conclusão

A implementação da suíte de testes de integração para o sistema AeroLog permitiu validar de forma robusta o comportamento do software em cenários próximos do ambiente real de produção.

O principal desafio técnico identificado residiu no isolamento e consistência do banco de dados entre as execuções dos testes. Em testes de integração sem *mocks*, registros inseridos por um cenário podem poluir o estado do banco e causar falsos negativos ou falsos positivos em testes subsequentes.

Para contornar esse desafio, utilizou-se o recurso de *fixtures* do pytest (conftest.py) integrado ao módulo tempfile do Python. Essa estratégia garante que cada função de teste execute sobre um arquivo de banco de dados SQLite temporário e isolado, criado do zero antes da execução e destruído imediatamente após a sua finalização.

Como consideração final, a obtenção de 100% de cobertura de código aliada a 100% de aprovação nos testes de integração garante que o sistema atende rigorosamente às regras de negócio operacionais, promovendo a segurança necessária para operações de transporte aéreo offshore.